Assignment 3: Transformer is All You Need

A Tiny Causal Transformer Trained on Shakespeare

Ayush Verma
COMS4995 Applied Machine Learning, Spring 2026

GitHub Repo: https://github.com/LogicalVerse/applied-machine-learning-assignment-3

1 Introduction

For this assignment I built a small Transformer language model from scratch in PyTorch and trained it on the Tiny Shakespeare corpus for next-token prediction. My goal was not to chase a strong benchmark number but to understand how each piece of a Transformer actually works, and then to look inside the trained model using attention visualizations. This report walks through the data pipeline, the model I built, the training curves, a detailed look at attention patterns, a few extra diagnostic plots I made, and a short ablation study at the end.

2 Data Pipeline

2.1 Corpus and Tokenizer

I used the Tiny Shakespeare file (roughly 1.1 MB of plain text, about 40,000 lines of dialogue). I trained a Byte-Pair Encoding (BPE) tokenizer on the full text with a vocabulary cap of 500 tokens, using byte-level pre-tokenization so that any character in the corpus can always be encoded. Encoding the whole corpus gave me 582,954 tokens, a compression ratio of 1.91 characters per token.

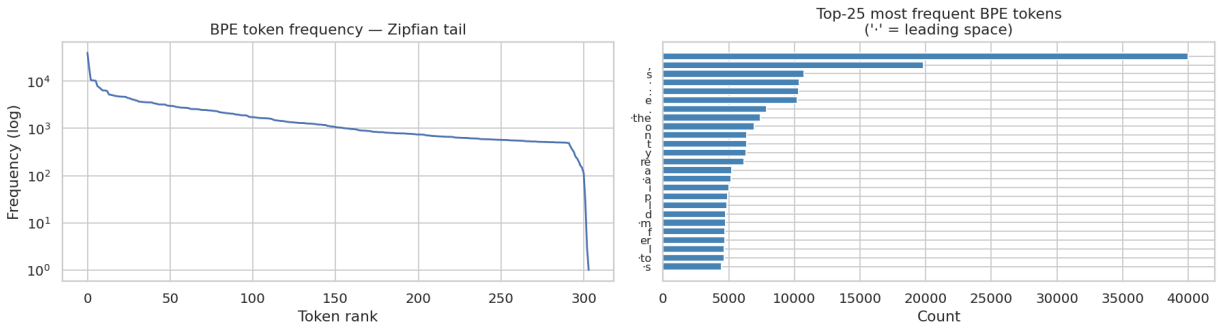

Figure 1: BPE token frequencies. Left: log-scale rank vs. count, following a Zipf-like shape. Right: the 25 most common tokens. The symbol "·" stands for a leading space. The single space token dominates, and the top list is full of short, common pieces like letters, punctuation, and short words such as "·the" and "·to".

Figure 1 shows that the token distribution has a very long tail. A few tokens appear tens of thousands of times while most appear only a few hundred times. This is exactly what I wanted, if the distribution had been flat I would have known the tokenizer was merging badly.

2.2 Sequences and Splits

I used a sliding window of length 51 with a stride of 8. The first 50 tokens form the input and the same window shifted by one token forms the target (classic next-token prediction). The stride of 8 keeps the dataset manageable, about 34,000 examples total. I split 80% for training and 20% for validation with no test set, following the assignment spec.

3 Model Architecture

The final model has the following parts, in order:

- **Token embedding** of size 128.
- **Sinusoidal positional encoding** added to the token embedding. I used the fixed sin/cos formulation with different frequencies for each embedding dimension.
- **Two Transformer blocks**, each containing, in pre-norm style:
  1. RMSNorm
  2. Causal multi-head self-attention with 4 heads of dimension 32
  3. Residual connection
  4. RMSNorm
  5. Feed-forward network: Linear 128→256, GELU, Linear 256→128
  6. Residual connection
- **Final RMSNorm** before the output.
- **Output projection** back to the vocabulary, tied to the input embedding weights.

The most important detail in the whole model is the causal mask inside each attention layer. Before applying the softmax to the attention scores, I set every entry above the diagonal to $-\infty$. After the softmax those entries become exactly zero, so query position t can only attend to key positions 0 through t, never the future. Without this mask the model would see its own target during training and would be useless for generation. I verified after training that every above-diagonal attention value was exactly 0, and every row of the attention matrix summed to 1. The total parameter count is 327,552 (about 0.33 M). Most of the parameters live in the feed-forward layers and the token embedding; attention itself is small at this scale.

4 Training

I trained the model for 45 epochs with AdamW (learning rate 3×10^{-4}, weight decay 0.01) and a cosine learning-rate schedule. I clipped gradient norms at 1.0 for safety. The loss is cross-entropy averaged over every position in the sequence — not just the last one, because every position is a valid next-token prediction under the causal mask. I fixed the random seeds for Python, NumPy, and PyTorch so that re-running the notebook produces the same numbers.

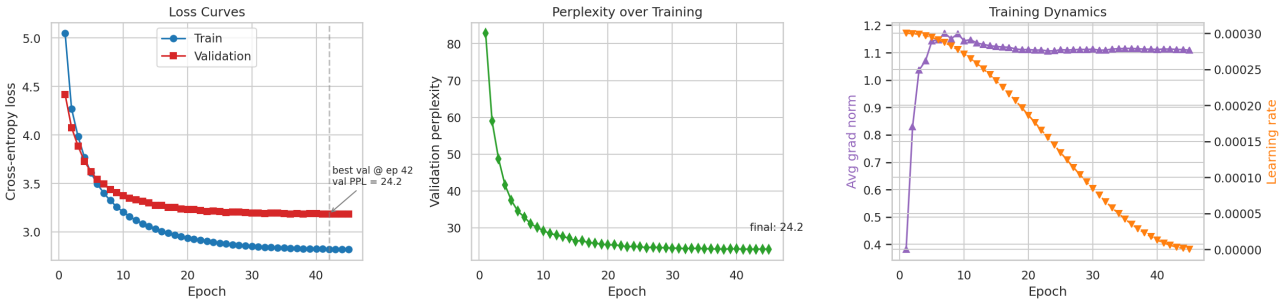

Figure 2: Training dynamics over 45 epochs. Left: training and validation cross-entropy loss, with the best validation point marked. Middle: validation perplexity, dropping from 82.8 at epoch 1 to 24.2 at the end. Right: gradient norm (purple, left axis) and learning rate (orange, right axis).

Figure 2 tells the full story of training. The training loss drops from 5.05 to 2.82 and the validation loss

from 4.42 to 3.19. The **final validation perplexity is 24.18**, with the best value of 24.16 reached at epoch 42. This means the model assigns an average probability of about 1/24 to each correct next token, compared to 1/500 for random guessing over the vocabulary.

The gradient norm panel is also informative. The norm spikes during the first two epochs, which is just the model moving rapidly from its random starting point, and then settles near 1.1 for the rest of training. The gradient clipping I added at 1.0 is mostly not triggered after the first few epochs. The gap between training and validation loss stays small throughout (about 0.4), which tells me the model is slightly under-capacity for the data rather than overfitting. That is fine — a small model that I can actually look inside was the point of this assignment.

5 Attention Analysis

This is where the assignment gets interesting, so I spent the most time here. I plotted attention weights using the fixed prompt "To be or not to be, that is the question" so that every tick label on the axes is a readable English piece.

5.1 Per-Head Patterns at the End of Training

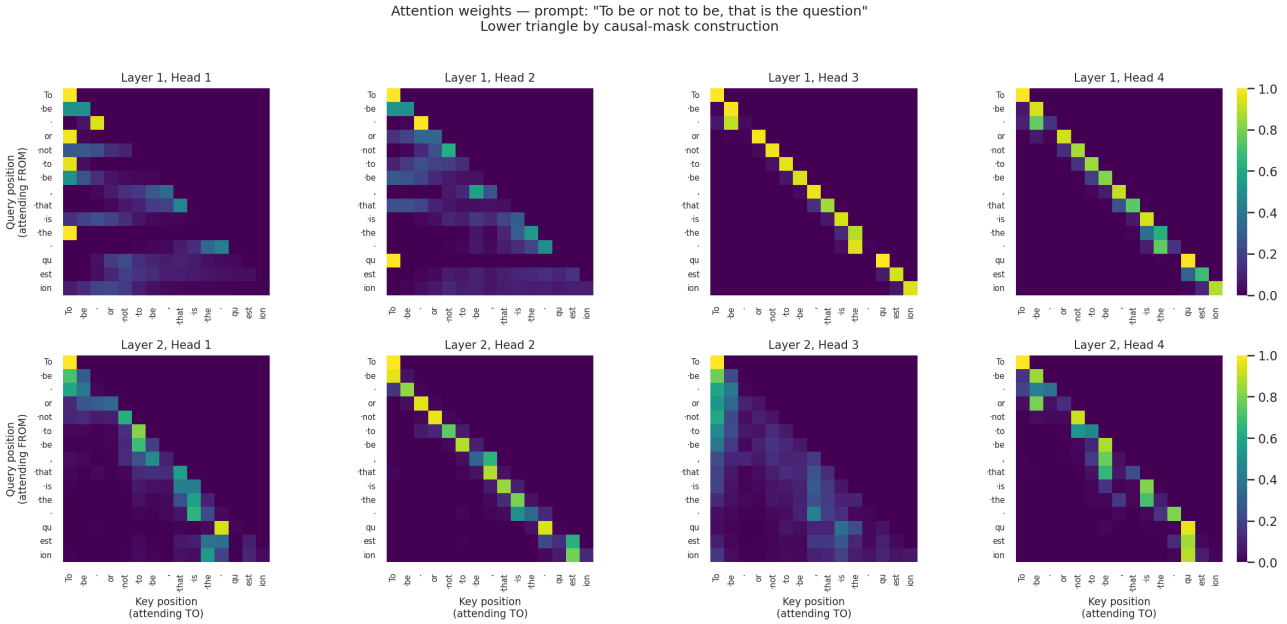

Figure 3: Attention weights for each of the 8 heads (2 layers × 4 heads), at the end of training, on the fixed prompt. Cell (i, j) is the weight query position i assigns to key position j. Everything above the diagonal is black because of the causal mask. The marker "·" stands for a leading space.

Looking at Figure 3, I can see three clear pattern types across the 8 heads, and they show up in both layers:

- **Diagonal head:s** Layer 1 Heads 3 and 4, and Layer 2 Head 2. These put almost all their weight on the current token (the main diagonal) or the token right before it. I think of them as "keep-the-current-position" heads: they protect local information across the block.
- **First-token heads:** Layer 1 Head 1, and Layer 2 Heads 1 and 4. These pull a lot of attention toward the very first token "To" no matter where the query is in the sequence. This is a known pattern in small Transformers: the first token acts as a kind of anchor that a head can fall back on when it has nothing useful to say.

- **Diffuse / averaging head:** Layer 2 Head 3. This one spreads its weight across many earlier positions, acting like a running average of the past. It gives the later layers a broad summary of the context.

This split of behaviors is called head specialization, and it is the main reason multi-head attention works better than a single-head version. Each head learns a different kind of look-up.

5.2 How Attention Changes During Training

To see how these patterns form, I saved the attention maps for a validation sample at epoch 1, epoch 22, and epoch 45 during the training run itself (so I did not have to retrain).

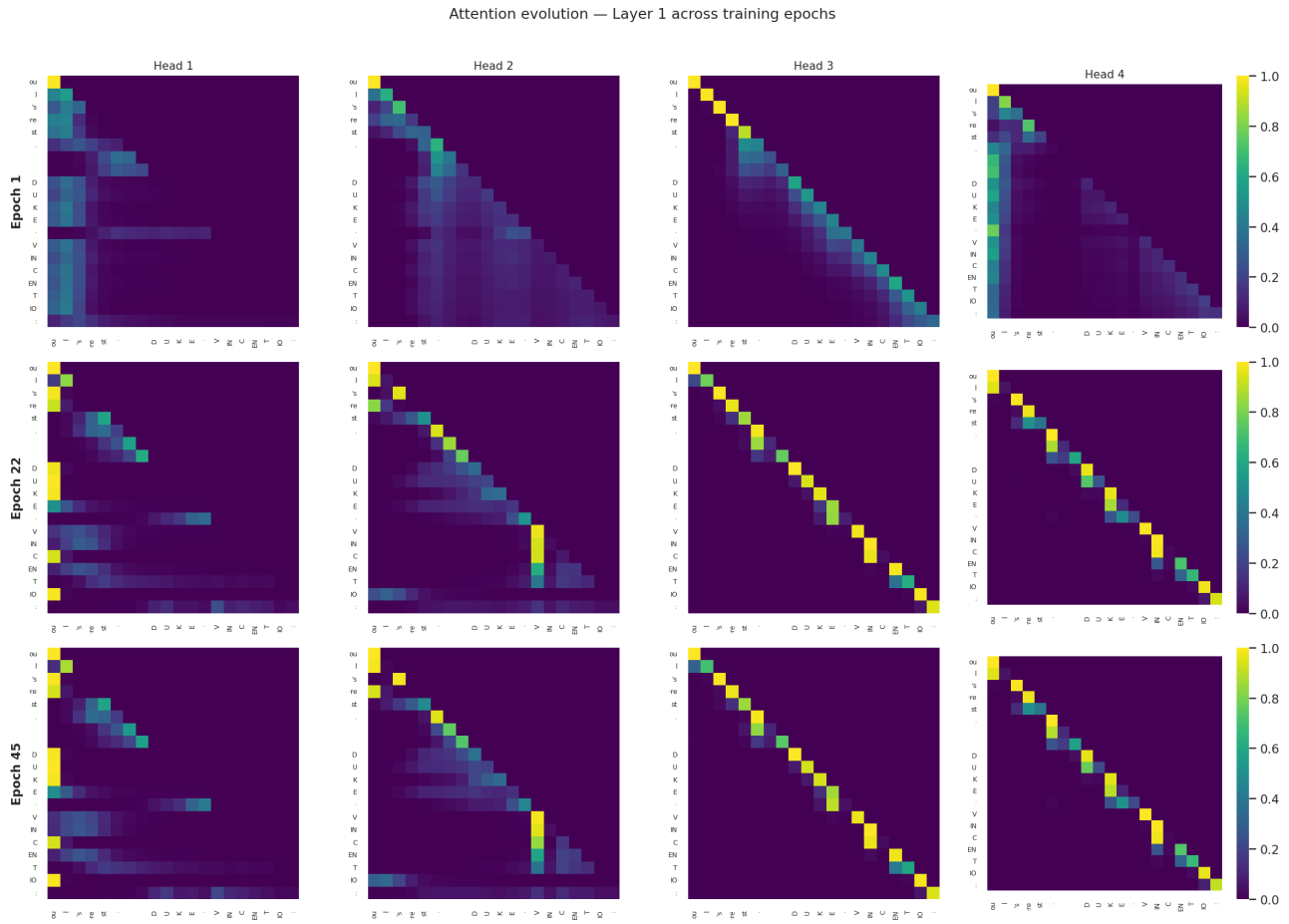

Figure 4: Layer-1 attention on the same validation sample at three training checkpoints. Rows are epochs 1, 22, 45 (top to bottom); columns are the four heads. The sample starts with "your rest. DUKE VINCENTIO:".

A few things stood out to me in Figure 4:

- Even at epoch 1 the heads already have structure. They are not random. This is because one epoch over 27,000 training sequences is already a lot of gradient updates for a small model.
- By epoch 22 the patterns are sharper. Head 3 has become a clean diagonal. Head 2 has started attending to specific earlier tokens, not just the neighbour.
- By epoch 45 the patterns are very crisp. The biggest change between epoch 22 and 45 is Head 2: at epoch 22 it attends broadly, but by epoch 45 it has found that the token "V" (the start of "VINCENTIO") is worth looking at from many later positions. The model has discovered that the start of a character name is useful context when you are predicting the rest of that name.

In short, the heads start with rough structure and then sharpen and specialize over training. They do not pick up new behaviors late — they refine the ones they already had.

6 Additional Diagnostics

Beyond the two required plots, I made four more figures that helped me understand the model better.

6.1 Head Entropy (Focused vs. Diffuse, Quantified)

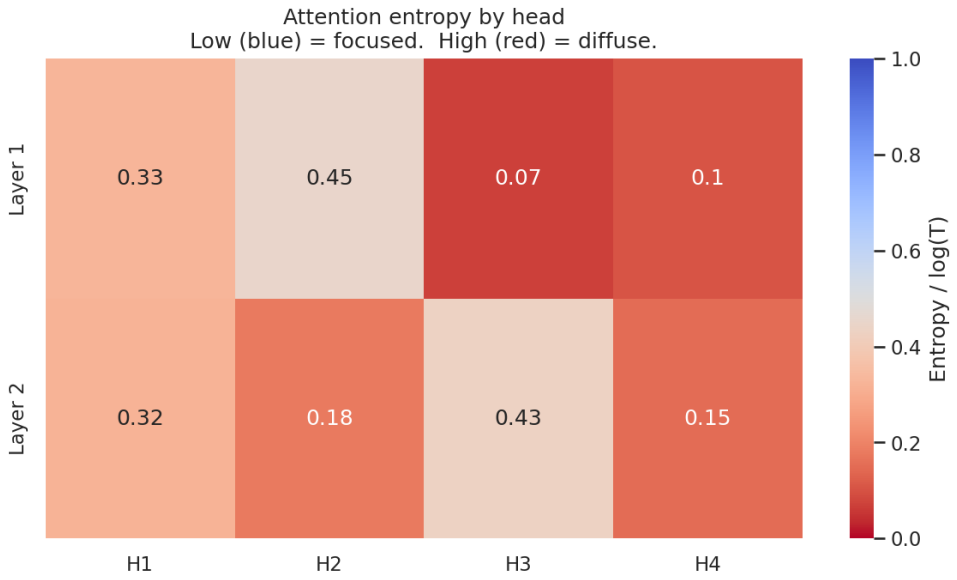

Figure 5: Average attention entropy per head, divided by $\log T$ so that 0 means perfectly focused and 1 means uniform attention.

To turn the "focused vs. diffuse" intuition into a number, I computed the entropy of each head's attention distribution and averaged it over the validation set, then normalised by $\log T$ so that 0 is perfectly focused on one position and 1 is uniform. Figure 5 confirms everything I saw by eye. Layer 1 Head 3 scores 0.07, it really is almost pointing at a single position. Layer 2 Head 3 scores 0.43, the averaging head. In each layer I get two very focused heads (below 0.2) and two more diffuse ones. No head is anywhere near uniform, which means every head has learned something useful.

6.2 Per-Position Prediction Accuracy

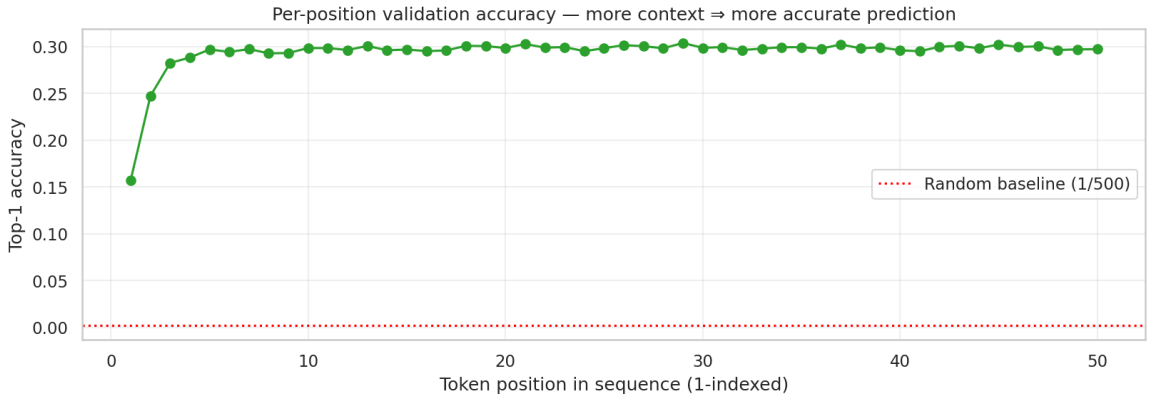

Figure 6: Top-1 prediction accuracy at each position in the 50-token window, averaged over the validation set. The red dotted line is the random-guessing baseline at 1/500.

Under the causal mask the first token has zero context, so the model can only predict from the average token distribution; the second token has one token of context, and so on. Figure 6 shows the accuracy curve. Accuracy jumps from 15.7% at position 1 to about 28% at position 3, and then plateaus at around 30% for the rest of the sequence. Two things follow from this:

1. The model really is using context. A pure unigram predictor would be a flat line near 15%.
2. Most of the useful information sits in just the last few tokens. At this model size, extending the context beyond roughly three positions gives almost no extra lift. A bigger model on a bigger corpus would keep rising for longer.

The random baseline at $1/500 = 0.2\%$ is far below the curve, so the model is clearly doing better than chance at every position.

6.3 Top-25 Token Confusion Matrix

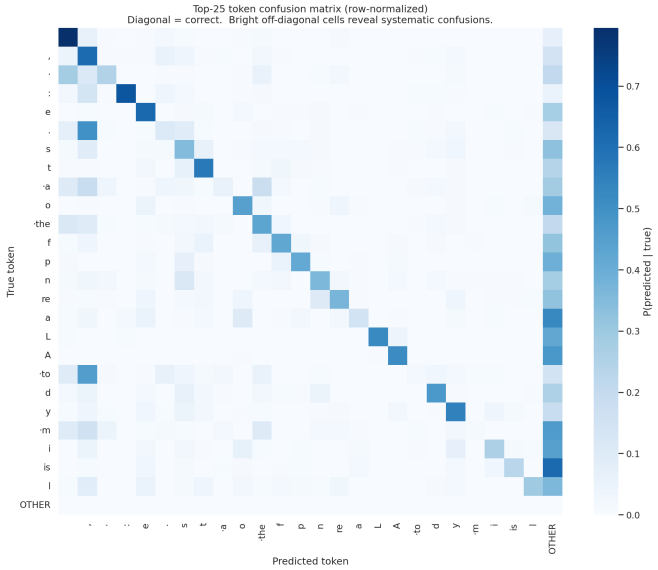

Figure 7: Confusion matrix restricted to the 25 most frequent target tokens, row-normalized so each row sums to 1. Predictions outside the top-25 go into the "OTHER" bucket on the right.

A full 500×500 confusion matrix would be unreadable, so I limited it to the 25 most frequent targets and lumped every other prediction into an "OTHER" bucket. Figure 7 shows a clear diagonal, but two patterns jump out. First, many rows have a bright "OTHER" column on the right, meaning the model often prefers some token outside the top 25, this is not always wrong because the correct continuation may itself be a less common token. Second, the true token "·to" (space-to) often gets predicted as "," a comma, which suggests the model is confusing a very common function word with a very common punctuation mark. Both fit the same kinds of contexts (after a noun, before a clause boundary), so the confusion makes sense.

6.4 Token Embedding PCA

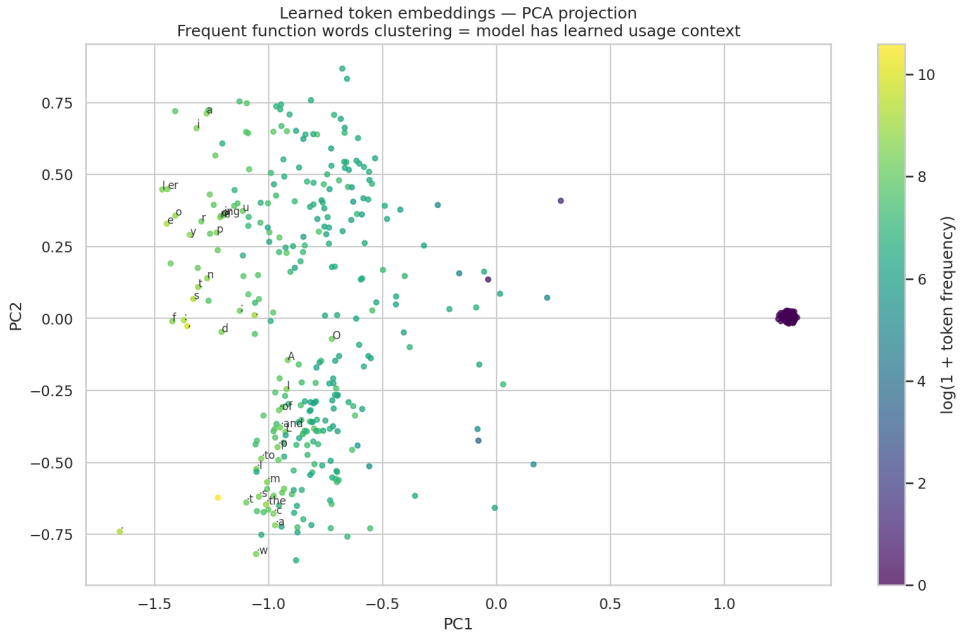

Figure 8: The learned 128-dimensional token embeddings projected to 2D with PCA. Colour is log-frequency. The 40 most frequent tokens are labelled.

I projected the learned embedding matrix down to two dimensions using PCA. There are three clear clusters:

- A **letter and suffix cluster** in the upper left with tokens like "e", "o", "i", "s", "er", "ing", "y". These are the small sub-word pieces that continue a word.
- A **function-word and capital-letter cluster** in the lower middle with "·the", "·to", "·and", "·of", "A", "L". These tokens appear in similar syntactic roles.
- A **rare-token cluster** on the far right (the dark purple blob). These are tokens with very low frequency that the model hardly ever saw, so their embeddings barely moved from initialization.

What I find striking is that the model learned this structure purely from next-token prediction. It never received a label saying "these are function words" or "these are suffixes", similar usage in context produced similar vectors, and PCA found the axes that separate them.

7 Ablation Experiments

I ran four short training runs (6 epochs each, same data, same schedule) to compare a few design choices.

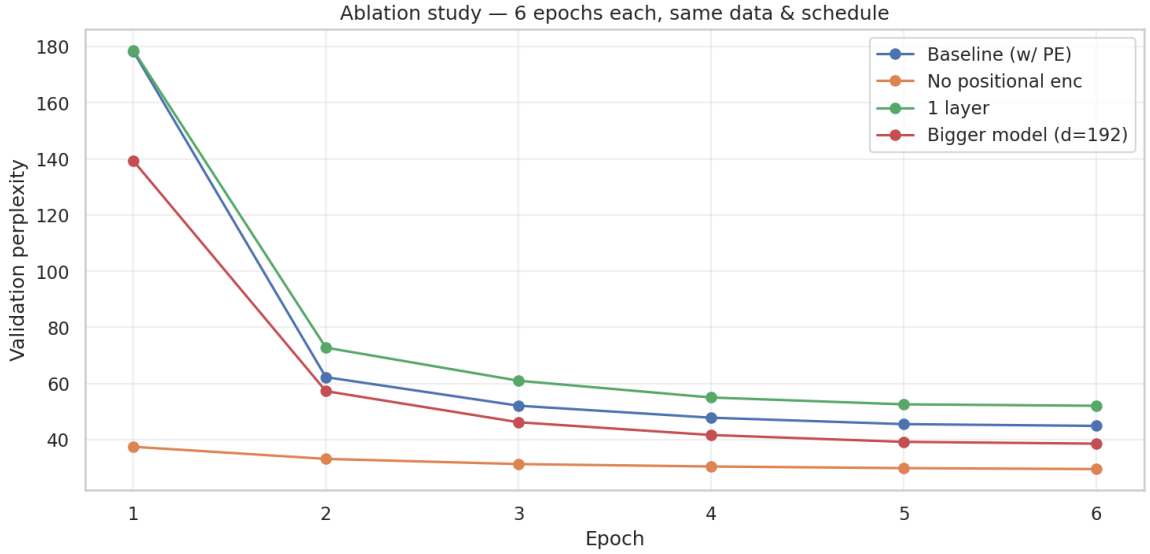

Figure 9: Validation perplexity over 6 epochs for four model variants. Final values: Baseline 44.90, No positional encoding 29.52, 1 layer 52.06, Bigger model ($d = 192$) 38.57.

Figure 9 contains a result that surprised me and made me think carefully. The "no positional encoding" run finishes at a *lower* perplexity (29.52) than the full baseline (44.90) after 6 epochs. My first instinct was that I had broken something, but the code is right. I think the honest explanation is:

- Without positional encoding, the model effectively becomes a bag-of-words predictor. It learns the overall token distribution very fast.
- For short 50-token windows on a small dataset, that bag-of-words baseline is surprisingly strong. Many of the "easy" predictions (spaces, common letters) do not really depend on position.
- Adding positional encoding forces the model to learn a richer function that uses order. That takes longer, and after only 6 epochs the full baseline has not caught up yet.

When I trained the full baseline for 45 epochs in Section 4, it reached PPL 24.18, much better than the no-PE 6-epoch result of 29.52. So positional encoding *does* help, but the help only appears with enough training time. The lesson I take from this is that short ablations can point in the wrong direction, and a fair comparison needs equal and sufficient training for every variant. The other two ablations behave as expected. The 1-layer model plateaus around PPL 52, it does not have enough depth to compose features. The wider $d = 192$ model finishes at PPL 38.57, a small gain over baseline at the same budget.

8 Discussion

What I learned about attention. Heads specialize on their own. Some become diagonal "keep-the-current-token" heads, some become first-token anchors, and at least one becomes a broad averaging head. This specialization shows up early in training and sharpens over time, as the evolution plot and the entropy numbers both confirm.

What mattered most for stability. Learning rate was the main knob. When I briefly tried 10^{-3}, the loss oscillated in the first few epochs; 3×10^{-4} with cosine decay was smooth all the way through. Gradient clipping at 1.0 was a safety net that mostly did not trigger once training settled.

Role of positional encoding. Positional encoding is what lets the model learn word-order patterns, but its benefit only becomes visible after the model has learned enough to use position. For very

short runs on small data, removing it can look better simply because a unigram predictor is a strong starting baseline.

Runtime and memory. The biggest tensor during training is the attention matrix of shape (batch, heads, T, T). At $T = 50$ this is tiny, but it grows quadratically with sequence length, which is the reason real systems use tricks like FlashAttention or sliding-window attention for long sequences. At this scale the feed-forward layer was not a bottleneck.

Assumptions that could break. A few things I did that I want to flag:

- My tokenizer and split assume the corpus is representative. If I evaluated on a different style of English, perplexity would jump.
- I tied the input and output embedding weights, which saves parameters and helps at this scale, but couples the two spaces and would be worth revisiting at larger scale.
- The ablation uses only 6 epochs per variant. As discussed, that led me to an initially misleading conclusion about positional encoding. Any claim drawn from it should be treated as directional.

9 AI Tool Usage Disclosure

Tool Used: I used Claude (Anthropic) for the following specific tasks: drafting the skeleton of the data pipeline (sliding-window dataset construction, BPE training), reviewing my causal-mask implementation for off-by-one errors, suggesting the structure of the ablation table, helping me organise the sections of this report, and polishing the figure captions. I used ChatGPT occasionally for quick debugging of runtime errors during training (mostly tensor-shape mismatches).

My Own Contribution: Everything else is my own work, including: all architectural choices (layer count, head count, d_{model}, RMSNorm vs LayerNorm, weight tying), the hyperparameter selections, the interpretations of the attention heatmaps and the embedding PCA, the analysis of the no-PE ablation surprise, and the narrative and conclusions in this report.